



CNN-Based Real-Time Driver Drowsiness Detection using Eye State Analysis

Ayush Mittal (23/CS/106)
Chitranshu Mahaur (23/CS/113)

Authors of Base Paper:

Ruben Florez , Facundo Palomino-Quispe, Roger Jesus Coaquira-Castillo , Julio Cesar Herrera-Levano , Thuanne Paixão and Ana Beatriz Alvarez

The Critical Need for Driver Alertness Monitoring

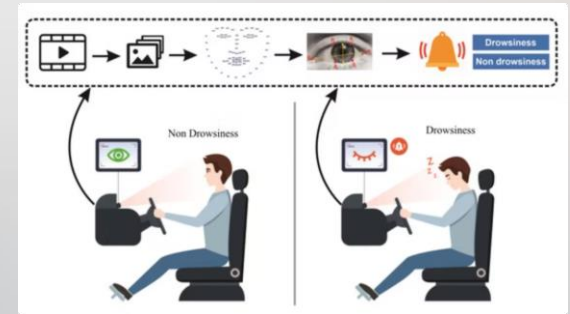
- Driver drowsiness causes **~20% of road accidents worldwide**
- Leads to **slow reaction time, poor judgment, and micro-sleeps (3–5 sec)**
- High risk during **long drives, night travel, and monotonous roads**
- Existing systems **lack real-time detection** or give **too many false alarms**
- Need for a **smart, reliable, real-time monitoring system**



Proposed Solution

- Detect driver drowsiness using **eye state (open/closed)**
- Focus on **eye region (ROI)** for accurate analysis
- Use:
 - **Convolutional Neural Networks (CNNs)** for classification
 - **Real-time video stream** for continuous monitoring
- System Workflow:
 - Capture video → Extract eye region → Classify state
- Alert Mechanism:
 - If probability of drowsiness > 95
 - If eyes remain closed for > 300 ms

System triggers **warning alarm**



System Pipeline

The system works in 6 major stages:

1. **Data Collection:** Uses NITYMED dataset (real driving videos at night)
2. **Frame Extraction:** Videos → frames (25 FPS)
3. **Face & Eye Detection:** Uses MediaPipe Face Mesh
Detects 468 facial landmarks
4. **ROI (Region of Interest) Extraction:** Focuses only on eye region Custom algorithm improves accuracy even when head moves
5. **CNN Training Models trained to classify:**
 - 😊 Not drowsy
 - 🌀 Drowsy
6. **Real-Time Detection:**
 - If: Eye closed probability > 95% AND time > 300 ms → 🚨 Alert triggered

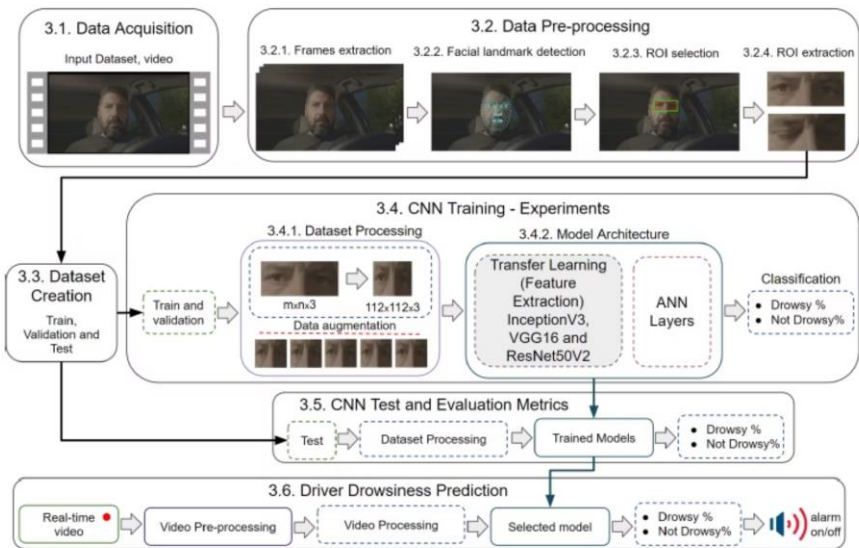
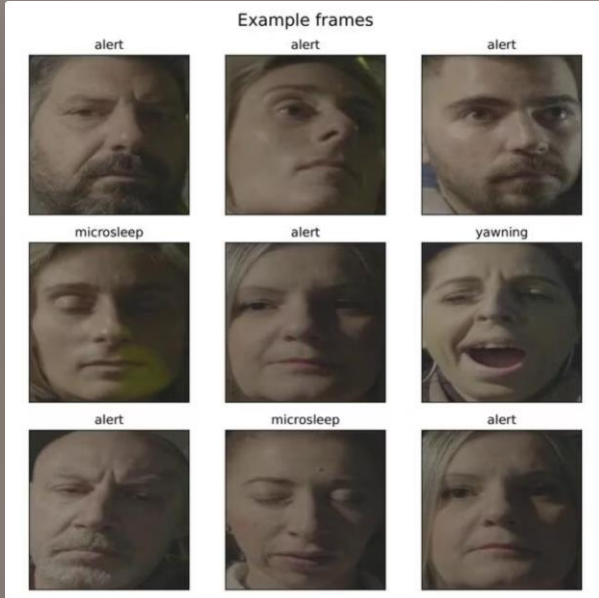


Figure 1. Methodology for the detection of driver drowsiness.

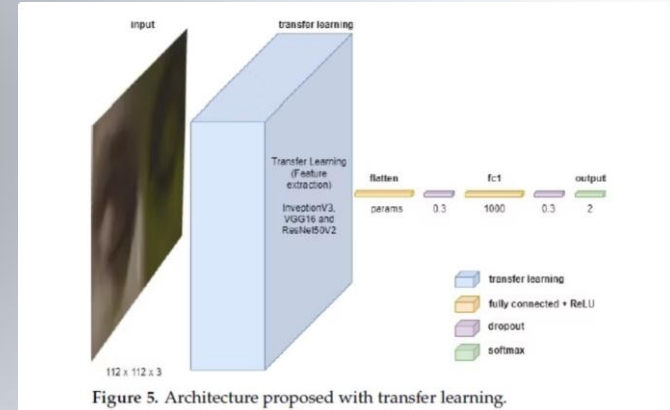
Dataset



- Dataset used: NITYMED: <https://datasets.esdalab.ece.uop.gr/download-files/>
- Consist of 130 Night-time driving videos
- Captured in **real driving conditions (night, fatigue)**
- Total Images: **30000**
- Classes:
 - **DROWSY**
 - **NOT DROWSY**
- Data Split:
 - **Training:** 80%
 - **Validation:** 20%

Model Architecture

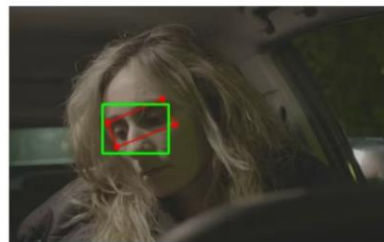
- Used **Transfer Learning** with pre-trained CNN models:
 - InceptionV3
 - VGG16
 - ResNet50V2
- Custom Layers Added:
 - **Dense Layer:** 1000 neurons (ReLU)
 - **Dropout:** 30% (prevent overfitting)
 - **Output Layer:** Softmax (2 classes)
- Task:
 - Binary Classification → **Drowsy / Not Drowsy**



Preprocessing

- **Frame Extraction**
Convert real-time video into individual frames for analysis
- **Face & Landmark Detection**
Use MediaPipe to detect **468 facial landmarks**
- **Eye Region Extraction (ROI)**
Isolate eye area from landmarks for focused drowsiness detection
- **Image Resizing**
Resize eye images to **112 * 112 pixels** for uniform input
- **Normalization**
Scale pixel values to **0–1 range** for faster and stable training
- **Noise Reduction**
Remove irrelevant background information to improve accuracy

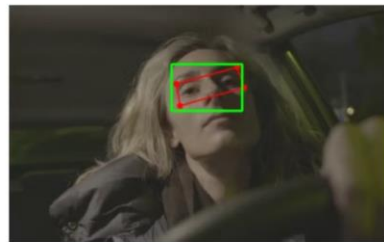
Helps in **reducing complexity, improving model performance, and ensuring consistent inputs.**



(a)



(b)



(c)



(d)

Training Details

- **Optimizer**

Used **Adam optimizer** for efficient and adaptive learning

- **Epochs**

Model trained for **30 epochs** to ensure proper convergence

- **Batch Size**

Batch size of **32** for balanced training speed and performance

- **Loss Function**

Used **categorical cross-entropy** for binary classification

- **Data Augmentation**

Applied techniques like:

- Rotation
- Horizontal flipping
- Zooming
 - 👉 Helps increase dataset diversity and prevent overfitting

- **Regularization**

Used **Dropout (30%)** to reduce overfitting

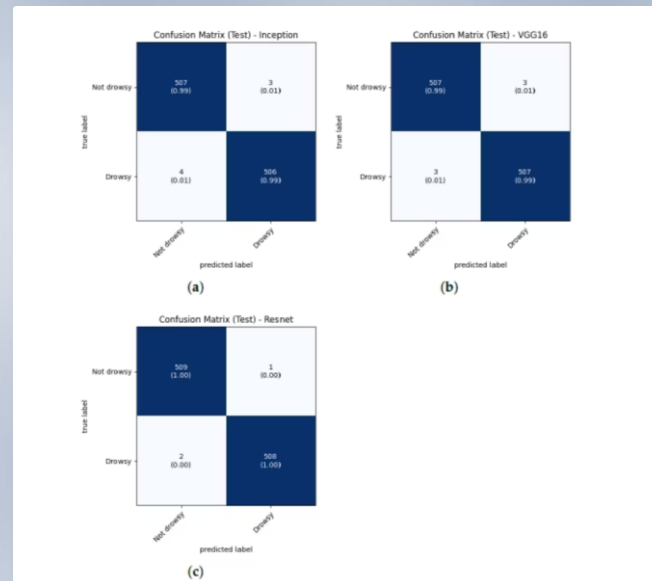
```
Epoch 1/10
750/750 [=====] - 4059s 5s/step - loss: 0.0810 - accuracy: 0.9801 - val_loss: 0.2202 - val_accuracy: 0.9355
Epoch 2/10
750/750 [=====] - 1937s 3s/step - loss: 0.0289 - accuracy: 0.9909 - val_loss: 0.3327 - val_accuracy: 0.9110
750/750 [=====] - 2320s 3s/step - loss: 7.0524 - accuracy: 0.5389 - val_loss: 0.6706 - val_accuracy: 0.5048
Epoch 5/10
750/750 [=====] - 1949s 3s/step - loss: 1.2797 - accuracy: 0.6047 - val_loss: 0.6960 - val_accuracy: 0.5230
Epoch 6/10
750/750 [=====] - 2333s 3s/step - loss: 0.7163 - accuracy: 0.6913 - val_loss: 0.5476 - val_accuracy: 0.6603
Epoch 7/10
750/750 [=====] - 1708s 2s/step - loss: 0.5191 - accuracy: 0.7441 - val_loss: 0.5280 - val_accuracy: 0.7062
Epoch 8/10
750/750 [=====] - 1763s 2s/step - loss: 0.2668 - accuracy: 0.9110 - val_loss: 0.4877 - val_accuracy: 0.8245
Epoch 9/10
750/750 [=====] - 2503s 3s/step - loss: 0.1426 - accuracy: 0.9527 - val_loss: 0.4808 - val_accuracy: 0.8433
Epoch 10/10
750/750 [=====] - 2765s 4s/step - loss: 0.1518 - accuracy: 0.9550 - val_loss: 0.4194 - val_accuracy: 0.8522
C:\Ayush work\College_work\DL project\dl_env\lib\site-packages\keras\src\engine\training.py:3080: UserWarning: You are saving your model as an
HDF5 file via `model.save()`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_m
odel.keras')`.
  saving_api.save_model(
(dl_env) PS C:\Ayush work\College_work\DL project> python -u "c:\Ayush work\College_work\DL project\models\realtime.py"
pygame 2.6.1 (SDL 2.28.4, Python 3.10.0)
```


Evaluation Metrics

- **Accuracy**
Overall correctness of the model
- **Precision**
How many predicted drowsy cases are actually correct
- **Recall**
Ability to correctly detect **actual drowsy cases**
- **F1-Score**
Balance between precision and recall

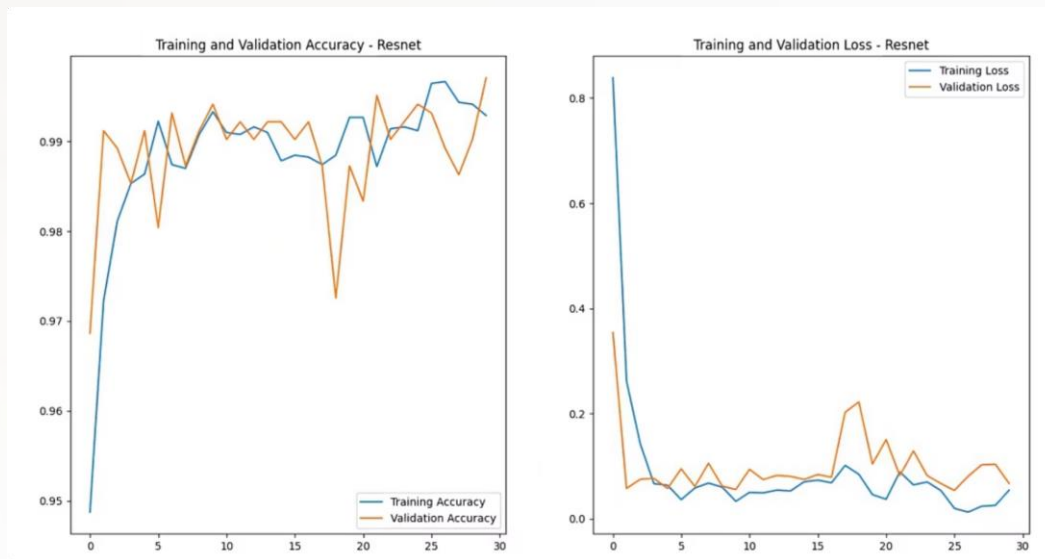
Key Focus

- **Minimize False Negatives**
(Missing a drowsy driver can lead to accidents 🚗)



Results

- Model performance evaluated on test dataset
- **Accuracy Achieved:**
 - InceptionV3 → **99.31%**
 - VGG16 → **99.41%**
 - ResNet50V2 → **99.71%**
- **Best Performing Model:**
👉 **ResNet50V2** with highest accuracy
- **Observations:**
 - All models show **high accuracy (>99%)**
 - Transfer learning significantly improves performance
 - Model effectively distinguishes between **drowsy and alert states**
- **Inference:**
System is **highly reliable for real-time drowsiness detection**



Visualization (Grad-CAM)

- Uses **Grad-CAM** for model interpretability
- Highlights **important regions** in the image used for prediction
- Model mainly focuses on:
 - **Eye region** 👁
 - Detecting open/closed state
- Generates **heatmaps**:
 - ● High importance
 - ○ Low importance

Key Insight

- Helps understand **why the model predicts drowsy or not**
- Confirms that model is learning **relevant features (eyes)**

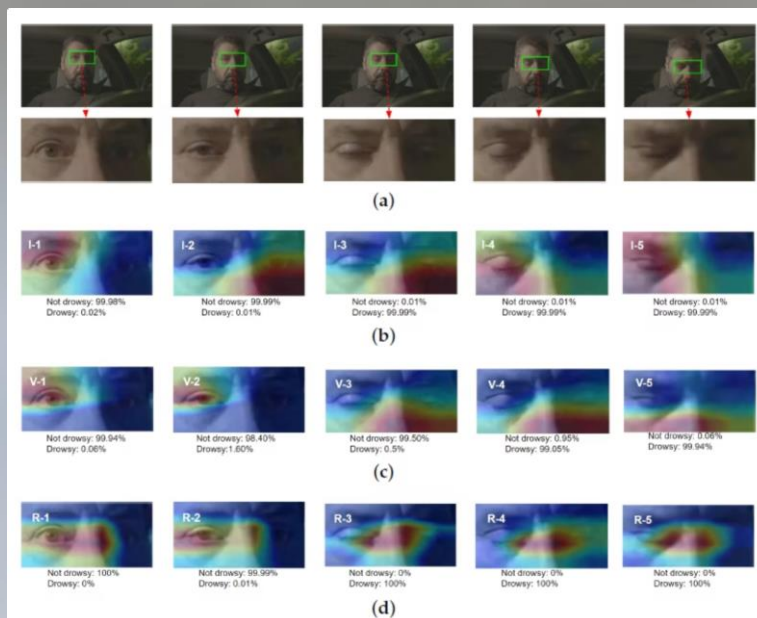


Figure 13. Visualized with Grad-CAM. (a) ROI extraction in 5 scenes. (b) Grad-CAM InceptionV3. (c) Grad-CAM VGG16. (d) Grad-CAM ResNet50V2.

Real-Time Detection

- Continuously captures and processes **live video stream** from the camera
- Detects face and extracts **eye region (ROI)** for each frame
- Uses trained **CNN model** to classify eye state as:
 - **Open (Alert)**
 - **Closed (Drowsy)**
- Tracks duration of eye closure across consecutive frames
- **Drowsiness Condition:**
 - If eyes remain closed for **> 300 ms**
 - System identifies driver as **drowsy**
- **Alert Mechanism:**
 - Triggers **audio alarm** 🔊
 - Provides instant warning to regain attention
- Displays real-time status: **Alert / Drowsy** on screen

👉 Ensures **continuous monitoring, quick detection, and immediate response for driver safety**



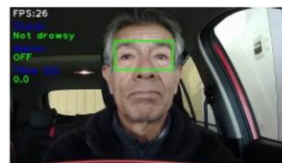
Limitations

- Performance depends on **lighting conditions** (low light may reduce accuracy)
- Requires **clear visibility of driver's face and eyes**
- May be affected by **occlusions** (glasses, hands, head movement)
- Accuracy can drop in **extreme real-world conditions**
- Requires **continuous camera access** for real-time monitoring

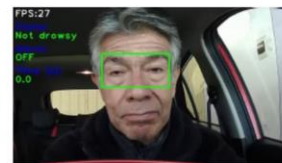


Future Work

- Use **Near-Infrared (NIR) Cameras**
Improve performance in **low-light / night conditions**
- Add **Yawning Detection**
Combine multiple fatigue indicators for better accuracy
- Deploy in **Embedded Systems**
Integrate into **real vehicle environments**
- Enhance Model:
 - Improve robustness
 - Optimize for faster real-time performance



(a) Frame 1



(b) Frame 2



(c) Frame 3



(d) Frame 4



(e) Frame 5



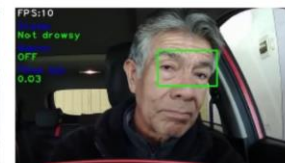
(f) Frame 6



(g) Frame 7



(h) Frame 8



(i) Frame 9

Conclusion

- Designed and implemented a **CNN-based driver drowsiness detection system** using eye state analysis
- Leveraged **transfer learning models (InceptionV3, VGG16, ResNet50V2)** to improve performance and reduce training time
- Achieved **very high accuracy (~99%)**, demonstrating strong model reliability
- **ResNet50V2** delivered the best results with highest accuracy and robustness
- System successfully classifies **drowsy and alert states** using real-time video input
- Detects prolonged eye closure (**> 300 ms**) to identify drowsiness effectively
- Provides **instant audio alerts**, helping the driver regain attention and avoid accidents

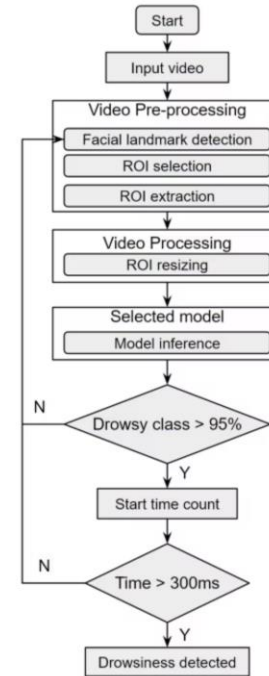


Figure 8. Drowsiness detection process flowchart.

THANK YOU

EFFORTS BY:

AYUSH MITTAL (23/CS/106)

CHITRANSHU MAHAUR(23/CS/113)

